

ENCRYPTED C2 VIA LEGITIMATE CHANNELS DETECTION

A Network-Level Sequential Threat Analysis System

PROJECT REPORT

Submitted by

Vandana S
ASI23CS189

Under the guidance of

Dr. NOOR MAHAMMAD SK



**Department of Computer Science and Engineering
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,
DESIGN AND MANUFACTURING KANCHEEPURAM
May 2026**

DECLARATION

I, the undersigned, hereby declare that the project report based on our topic “**ENCRYPTED C2 VIA LEGITIMATE CHANNELS DETECTION**”, submitted during the internship under the supervision of Head Of CSE Department **Dr. NOOR MAHAMMAD SK.**

With my signature, I certify that

- I have not manipulated any of the data or results.
- I have not committed any plagiarism of intellectual property. I have clearly indicated and referenced the contributions of others.
- I have explicitly acknowledged all collaborative research and discussions.
- I have understood that any false claim will result in severe disciplinary action.
- I have understood that the work may be screened for any form of academic mis-conduct.

PLACE: CHENNAI

DATE: _____

SIGNATURE

Vandana S

CERTIFICATE

This is to certify that the project report entitled

“ENCRYPTED C2 VIA LEGITIMATE CHANNELS DETECTION”

submitted by **Vandana S** (Roll No. **ASI23CS189**), a student of the **Department of Computer Science and Engineering, Adi Shankara Institute of Science and Technology, Kalady**, to the **Indian Institute of Information Technology, Design and Manufacturing (IIITDM), Kancheepuram**, is a bona fide record of the work carried out by her under my supervision and guidance as a part of the Internship Programme.

This report has not been submitted, either in whole or in part, to any other Institute or University for the award of any degree, diploma, or other academic qualification.

Dr. Noor Mahammad SK Associate Professor and Head Department of Computer Science and Engineering IIITDM Kancheepuram Chennai, Tamil Nadu

PLACE : Chennai

DATE : _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, **Dr. Noor Mahammad SK**, for his invaluable guidance, continuous support, and encouragement throughout the course of this internship. His expertise in machine learning, cybersecurity, and research methodology has been instrumental in shaping this work and facilitating its successful completion.

I am grateful to the **Indian Institute of Information Technology, Design and Manufacturing (IIITDM), Kancheepuram**, for providing an excellent research environment and the opportunity to undertake this internship. I also extend my thanks to the faculty members and researchers whose insights, suggestions, and discussions contributed significantly to this project.

I would like to express my sincere appreciation to the **Department of Computer Science and Engineering, Adi Shankara Institute of Engineering and Technology, Kalady**, for their constant support and encouragement throughout the internship period.

Finally, I would like to thank my family and friends for their unwavering motivation, understanding, and support, which enabled me to successfully complete this project.

ABSTRACT

For temporally extended attack patterns, such as beaconing for command-and-control (C2), flow-level statistical-based Network Intrusion Detection Systems (NIDS) are often unable to identify the malicious nature of individual flows. Sequence-based NIDS overcome this limitation but typically suffer from poor generalisation to unseen network environments. This work, entitled *A Network-Level Sequential Threat Analysis System*, addresses both challenges by developing and evaluating two independent classifiers and analysing their behaviour, failure modes, and generalisation capabilities without ensembling their predictions.

The first model is an XGBoost classifier that performs per-flow classification using 12 base traffic features and six engineered interaction terms, among which the protocol–destination-port interaction contributes the highest discriminative gain. The second model is a Temporal Convolutional Network (TCN) that processes windows of 20 consecutive flows per source IP address and incorporates seven within-window temporal features, including inter-arrival statistics, beaconing regularity, destination fan-out, and destination-port entropy. These sequence-level features are computed independently for each window and do not explicitly account for distribution shifts arising from previously unseen hosts.

Experiments conducted on the CTU-13 dataset show that XGBoost achieves a ROC-AUC of 0.9952, whereas the TCN attains a substantially higher malicious-class F1-score of 0.7364 compared to 0.5223 for XGBoost, demonstrating the effectiveness of temporal sequence modelling in detecting C2 beaconing behaviour that remains indistinguishable at the individual-flow level. A strict source-IP-disjoint train–validation–test partitioning strategy is employed to prevent identity leakage, and operating thresholds for both models are selected using an F2-optimal precision–recall criterion that prioritises malicious traffic recall.

Direct transfer of both models to the structurally different CICIDS-2017 dataset results in performance degradation due to domain shifts in traffic characteristics. Nevertheless, lightweight domain recalibration based on feature-separability reweighting and logistic/isotonic calibration substantially improves cross-dataset performance, increasing ROC-AUC to 0.9964 for the TCN and 0.9186 for XGBoost. Transferability analysis further shows that structural and behavioural features, such as destination fan-out and payload coefficient of variation, generalise well across datasets, whereas magnitude-based features, including total bytes and packet rate, exhibit limited transferability. These observations provide useful guidance for designing robust cross-domain NIDS.

Both models are integrated into a real-time deployment framework, *NIDS-Live*, implemented as a Flask-based dashboard that captures network traffic and independently reports flow-level and sequence-level maliciousness probabilities. The deployment maintains full inference parity with the offline training pipeline, enabling consistent behaviour between experimental evaluation and live operation.

Contents

DECLARATION	2
CERTIFICATE	3
ACKNOWLEDGEMENT	4
ABSTRACT	5
TABLE OF CONTENTS	7
LIST OF FIGURES	9
LIST OF TABLES	10
1 INTRODUCTION	11
1.1 Background and Motivation	11
1.2 Problem Statement	11
1.3 Research Objectives	12
1.4 Thesis Structure	12
2 LITERATURE REVIEW	13
2.1 Sequence-Based Models for Network Traffic Analysis	13
2.2 Cross-Dataset Generalization	14
2.3 Research Gaps	14
3 SYSTEM DESIGN AND METHODOLOGY	16
3.1 Dataset	16
3.2 Chunked Data Loading	17
3.3 Feature Engineering	18
3.3.1 Base Flow Features	18
3.3.2 XGBoost Interaction Features	19
3.3.3 Temporal Features Within Window (TCN Only)	20
3.4 Data Splitting and Scaling	22
3.5 Sequence Construction	22
3.5.1 XGBoost (Per-Flow Classifier)	24
3.5.2 Temporal Convolutional Network (Sequence Classifier)	24
3.6 Training Configuration	26

4	IMPLEMENTATION	27
4.1	Software Environment and Libraries	27
4.2	Memory-Bounded Pipeline Design	27
4.3	Training-Time Diagnostics	28
4.4	CICIDS-2017 Cross-Dataset Inference Pipeline	28
4.4.1	XGBoost Domain Recalibration	29
4.4.2	TCN Domain Recalibration	30
4.5	Independent Output Reporting	30
4.6	Live Deployment System: NIDS-Live	30
4.6.1	Architecture	31
4.6.2	Inference Parity	33
4.6.3	TCN Sequence Windowing in a Streaming Environment	33
4.6.4	Independent Model Operation	33
4.6.5	Operator Dashboard	34
5	RESULTS AND DISCUSSION	35
5.1	Experimental Setup	35
5.1.1	XGBoost (Per-Flow) Results on CTU-13	35
5.1.2	TCN (Sequence-Level) Results on CTU-13	36
5.2	Cross-Dataset Generalization on CICIDS-2017	38
5.2.1	Base (Non-Recalibrated) Performance	38
5.2.2	Recalibrated Performance	39
5.3	Ablation Study	40
5.3.1	Interaction Feature Ablation via XGBoost Gain Importance	40
5.3.2	Temporal Context Ablation: XGBoost as the Ablated Baseline	41
5.3.3	Cross-Domain Feature Transferability Analysis	42
5.4	Summary of Findings	42
6	CONCLUSION AND FUTURE WORK	45
6.1	Summary of Contributions	45
6.2	Limitations	46
6.3	Future Work	46

List of Figures

3.1	Distribution of the twenty most frequent traffic labels in the CTU-13 dataset. The dataset contains a mixture of background traffic, normal user activity, and botnet-generated traffic across thirteen capture scenarios. The logarithmic scale highlights the substantial class imbalance present in the dataset, where benign and background traffic categories greatly outnumber botnet-related flows.	17
3.2	Architecture of the proposed XGBoost-based flow-level classifier for encrypted command-and-control (C2) communication detection using the CTU-13 BinetFlow dataset. Each network flow is represented by raw BinetFlow attributes, transformed through feature engineering and interaction feature generation, normalized using a Yeo–Johnson power transformation with clipping, and subsequently classified by a gradient-boosted decision tree ensemble. The model outputs the probability that an individual network flow corresponds to malicious C2 activity.	23
3.3	Architecture of the proposed Temporal Convolutional Network (TCN) sequence classifier for encrypted command-and-control (C2) traffic detection using the CTU-13 BinetFlow dataset. The model processes windows of 20 consecutive flows augmented with within-window temporal features, followed by robust scaling, a TCN backbone composed of dilated residual blocks, and fully connected layers that estimate the probability of a malicious communication sequence. The right panel illustrates the residual block structure employed within the TCN backbone.	25
4.1	Class distribution of the CICIDS-2017 dataset used for cross-dataset evaluation. The left panel shows the binary class split, where benign traffic constitutes 79.12% of all flows and malicious traffic constitutes 20.88%. The right panel presents the distribution of attack categories on a logarithmic scale. DoS Hulk, PortScan, and DDoS attacks dominate the malicious traffic, while several attack categories such as Infiltration and Heartbleed are comparatively rare. The diversity of attack types and their distributional differences from CTU-13 contribute to the domain shift observed during cross-dataset evaluation.	29
4.2	Architecture of the NIDS-Live deployment system. The capture layer acquires network traffic through NFStream or Scapy-based packet capture engines and stores completed flows in a queue. The inference pipeline performs feature engineering, scaling, and independent XGBoost and TCN inference. Results are maintained within a shared pipeline state and exposed through a Flask REST API to a real-time web dashboard.	32

4.3	NIDS-Live operator dashboard during live network monitoring. The interface displays real-time flow statistics, model-specific alert counts, suspicious source IP rankings, live flow records, and an alert feed generated independently by the XGBoost and TCN detection pipelines. The dashboard also provides visibility into domain-shift warnings and capture engine status.	34
5.1	Confusion Matrix of the XGBoost classifier on the CTU-13 test dataset. The model correctly classified 5,903,690 benign flows and 4,896 malicious flows, while producing 7,045 false positives and 1,905 false negatives. The results correspond to a malicious-class recall of 71.99% at the selected operating threshold.	36
5.2	Confusion Matrix of the Temporal Convolutional Network (TCN) on the CTU-13 sequence-level test set. The model correctly classified 38,900 benign sequences and 619 malicious sequences, while producing 333 false positives and 119 false negatives. The resulting malicious-class recall was 83.88%, demonstrating the TCN's ability to capture temporal behavioural patterns associated with botnet activity.	38
5.3	Performance comparison of the recalibrated XGBoost and TCN models on the CICIDS-2017 dataset. The TCN consistently outperformed XGBoost across ROC-AUC, PR-AUC, F1-score, and F_2 -score, highlighting the benefit of temporal sequence modelling under domain adaptation. . .	40

List of Tables

5.1	Performance Before and After Domain Recalibration	39
5.2	XGBoost gain importance and ablation recommendations.	41
5.3	Temporal context ablation. XGBoost acts as the ablated baseline for the TCN. † F2-scores are from the CICIDS-2017 recalibrated evaluation. . .	42
5.4	TCN window-level feature transferability ranked by univariate ROC-AUC on CICIDS-2017.	43

Chapter 1

INTRODUCTION

1.1 Background and Motivation

One of the most significant threats for enterprise and infrastructure networks continues to be attacks from bots. When a host is infected, the malware establishes a command-and-control (C2) channel to a remote operator from which it receives instructions, exfiltrates information, or coordinates its participation in larger campaigns, such as a distributed denial-of-service (DDoS) attack. C2 traffic is intentionally designed to be indistinguishable from normal network traffic and is typically characterized not by bursty, high-volume communication, but rather by periodic, low-volume “beaconing” traffic.

Another privacy-preserving and computationally efficient alternative to deep packet inspection is the use of flow-level network telemetry, which collects aggregated statistics describing a connection between two hosts, such as duration, packet count, and byte count. However, flow-level detection presents its own challenges: malicious traffic constitutes only a small fraction of the total number of flows, individual malicious flows are not necessarily statistically anomalous, and the most indicative characteristic of malicious activity is often the temporal pattern exhibited across a sequence of flows rather than any single flow in isolation.

1.2 Problem Statement

Most of the current flow-based intrusion detection methods can be divided into two classes. Classical tabular models (e.g., Random Forest, XGBoost) are quick to learn decision boundaries on statistical features of each flow and are effective, but fail to model flow-to-flow time dependencies because each flow is scored individually. Sequence models (such as RNNs and TCNs) are capable of learning temporal and periodic structure between subsequent flows from the same host, but require more training time, are more sensitive to data imbalance, and are more susceptible to distribution shift when tested out of the training domain.

This project investigates whether a per-flow statistical model can reliably detect botnet activity in the CTU-13 dataset and whether a per-sequence temporal model can also reliably detect botnets in the CTU-13 dataset, and whether the statistical and temporal models would be able to detect botnets in a structurally different dataset (CICIDS-2017) without retraining from scratch. This project does not attempt to integrate the two models into a single ensembled score; instead, it keeps the two models independent and

separate so that the behavior, failure modes, and generalization of each model can be studied without one model obscuring the other.

1.3 Research Objectives

The main aims of this project are:

1. To develop a pipeline that is efficient in terms of memory usage to process the entire multi-scenario CTU-13 BinetFlow dataset using a chunked loading approach without having the entire dataset in memory.
2. To fit an XGBoost classifier on per-flow statistical and behavioural features (including engineered interaction terms) using an approach to correct class imbalance through `scale_pos_weight`.
3. To train a Temporal Convolutional Network (TCN) on fixed-length sequences of flows grouped by their source IP, using dilated causal convolutions to learn beaconing periodicity and other temporal patterns, together with focal loss to address class imbalance.
4. To test both models separately — without ensembling — using ROC-AUC, PR-AUC, and threshold-based classification on a validation split that is disjoint by source IP from the training data.
5. To evaluate cross-dataset generalization using both trained models on the CICIDS-2017 dataset through feature alignment and domain recalibration.

1.4 Thesis Structure

The rest of this report is structured as follows:

- **Chapter 2: Literature Review** — Analyses machine learning and deep learning methods used for flow-based botnet and intrusion detection.
- **Chapter 3: System Design and Methodology** — Describes the CTU-13 dataset, the chunked loading and feature engineering pipeline, the XGBoost model, the TCN architecture, and their training configurations.
- **Chapter 4: Implementation** — Discusses memory management strategies, sequence construction using HDF5 caching, and the CICIDS-2017 cross-dataset inference pipeline with domain recalibration.
- **Chapter 5: Results and Discussion** — Presents and discusses the independent performance of the XGBoost and TCN models on both CTU-13 validation data and CICIDS-2017.
- **Chapter 6: Conclusion and Future Work** — Summarizes the findings of the project and provides suggestions for future improvements and extensions.

Chapter 2

LITERATURE REVIEW

Traditionally, flow-based Network Encrypted C2 via legitimate channels detection) have relied on tabular machine learning models. To aid comparative assessment of botnet detection methods, Garcia et al. introduced the CTU-13 dataset as a mixture of real botnet, normal, and background traffic across thirteen different malware scenarios, demonstrating that botnet detection difficulty varies according to the command-and-control (C2) behavior of the malware. Tree-based ensemble methods such as Random Forest and Gradient Boosted Trees (e.g., XGBoost introduced by Chen and Guestrin) have been widely applied to such datasets because they are well suited to non-normally distributed flow features and scale efficiently to large tabular datasets.

However, these models classify each flow independently. As a result, they cannot directly represent behaviors that are only observable across a sequence of connections, such as periodic beaconing intervals or slowly evolving scanning activities. Their performance is also sensitive to class imbalance, which is often severe in botnet datasets because malicious flows constitute only a small fraction of total traffic. In most cases, this issue is addressed through cost-sensitive weighting rather than resampling, thereby preserving the original flow distribution.

2.1 Sequence-Based Models for Network Traffic Analysis

A sequence model is an ordered collection of values that captures dependencies between observations over time. For modeling temporal relationships in network traffic, researchers have employed recurrent architectures such as Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs), as well as more recent convolutional sequence models.

Bai et al. demonstrated that Temporal Convolutional Networks (TCNs) can outperform recurrent architectures in many sequence modeling tasks, or achieve comparable performance while providing faster training and avoiding the vanishing gradient problem associated with long sequences. This advantage arises from the use of dilated causal convolutions, which increase the receptive field of the network without requiring recurrent connections.

Within the network security domain, sequence models organized by source IP address or flow tuple have been used to identify periodic C2 beaconing activity. In such cases, the regularity of packet or flow arrivals may be more indicative of compromise than

the statistical characteristics of any individual flow. Consequently, temporal models can capture attack behaviors that are difficult to detect using per-flow statistical analysis alone.

The problem of class imbalance becomes even more challenging when constructing sequences. Malicious sequences are often rare, and naïve sequence generation strategies may produce training batches containing very few positive examples, leading to poor convergence during training. Focal Loss, originally proposed by Lin et al. for dense object detection, has since been adapted to imbalanced classification tasks including network intrusion detection. By down-weighting easily classified majority-class examples, Focal Loss enables the model to focus more effectively on difficult and minority-class samples.

2.2 Cross-Dataset Generalization

One of the most widely recognized challenges in Network Intrusion Detection System (NIDS) research is poor cross-dataset generalization. Models trained on one network capture dataset frequently experience performance degradation when evaluated on another dataset, even when both datasets represent similar attack categories. This phenomenon is commonly attributed to differences in network topology, traffic distributions, capture durations, and feature extraction methodologies, all of which contribute to domain shift between training and deployment environments.

Sharafaldin et al., while introducing the CICIDS-2017 dataset, highlighted that many earlier intrusion detection datasets focused on a limited set of attack types and often contained unrealistic traffic distributions. As a result, they recommended evaluating intrusion detection systems across multiple datasets to better assess their practical deployment capabilities.

2.3 Research Gaps

Based on the literature reviewed, the following research gaps are identified:

1. **No Independent Model Comparison:** Many existing systems combine statistical and temporal models through ensemble methods or score fusion. While this often improves overall performance, it obscures the individual strengths, weaknesses, and failure modes of each model. Limited research has evaluated per-flow statistical models and temporal sequence models independently on the same dataset.
2. **Memory Constraints at Scale:** Large datasets such as the complete CTU-13 corpus present significant memory challenges. Many published approaches do not adequately address the engineering requirements necessary for memory-efficient feature extraction and model training on resource-constrained systems.
3. **Insufficient Cross-Dataset Validation:** Many intrusion detection models are evaluated only on held-out portions of the same dataset used for training. Consequently, their ability to generalize to structurally different network environments remains insufficiently studied.

To address these challenges, this project trains XGBoost and Temporal Convolutional Network (TCN) models independently, without ensembling, using a chunked and

memory-bounded processing pipeline on the CTU-13 dataset. The generalization capability of both models is subsequently evaluated through a cross-dataset inference pipeline using the CICIDS-2017 dataset.

Chapter 3

SYSTEM DESIGN AND METHODOLOGY

3.1 Dataset

The system is trained on the CTU-13 dataset, a publicly available collection of captured botnet traffic represented in bidirectional NetFlow-like format (BinetFlow format) across thirteen different botnet scenarios. These scenarios contain a mixture of background traffic, normal traffic generated by legitimate hosts, and malicious traffic generated by infected hosts. Each flow is labeled according to the process responsible for generating the traffic.

In this project, all flows labeled as `Background`, `Normal`, or `BENIGN` are treated as benign flows (`is_malicious` = 0), while all remaining labels are treated as malicious flows (`is_malicious` = 1).

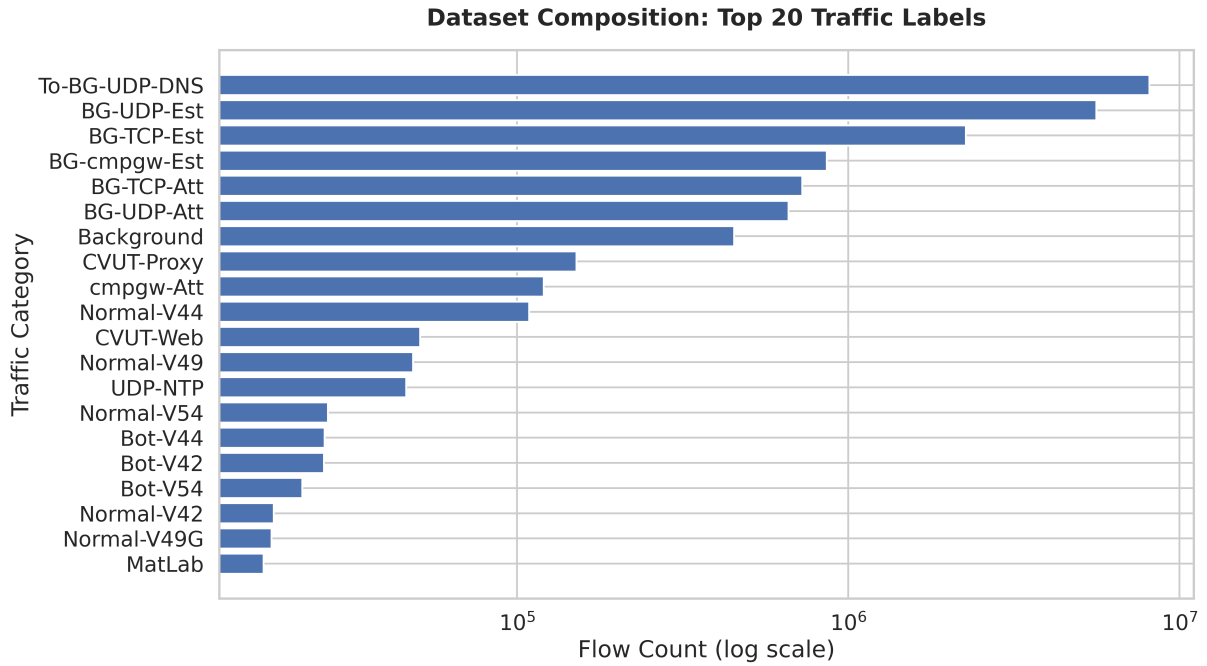


Figure 3.1: Distribution of the twenty most frequent traffic labels in the CTU-13 dataset. The dataset contains a mixture of background traffic, normal user activity, and botnet-generated traffic across thirteen capture scenarios. The logarithmic scale highlights the substantial class imbalance present in the dataset, where benign and background traffic categories greatly outnumber botnet-related flows.

3.2 Chunked Data Loading

All CTU-13 BinetFlow files are processed using a chunked loading strategy in which files are read in fixed-size windows of rows (`LOAD_CHUNK_ROWS = 500,000`) using the Pandas chunked CSV reader. This prevents the entire dataset from being loaded into memory simultaneously.

Only a subset of relevant columns is retained for each chunk:

- StartTime
- Dur
- Proto
- SrcAddr
- Sport
- Dir
- DstAddr
- Dport
- TotPkts

- TotBytes
- SrcBytes
- Label

Each chunk is additionally tagged with the scenario from which it originated before being appended to the working dataset. This approach ensures that peak memory consumption remains bounded regardless of the total dataset size.

3.3 Feature Engineering

3.3.1 Base Flow Features

The following behavioural ratio features are derived from the raw BinetFlow columns:

- **BytesPerPkt**

$$\text{BytesPerPkt} = \frac{\text{TotBytes}}{\text{TotPkts}}$$

- **PktRate**

$$\text{PktRate} = \frac{\text{TotPkts}}{\text{Dur}}$$

- **ByteRate**

$$\text{ByteRate} = \frac{\text{TotBytes}}{\text{Dur}}$$

- **SrcBytesRatio**

$$\text{SrcBytesRatio} = \text{clip} \left(\frac{\text{SrcBytes}}{\text{TotBytes}}, 0, 1 \right)$$

Categorical fields are label encoded (**Proto_enc**, **Dir_enc**). Source and destination ports are bucketed into six categories:

- Well-known ports (≤ 1023)
- Registered ports (≤ 49151)
- Ephemeral ports
- HTTP/HTTPS ports
- DNS ports
- Unparseable ports

These are represented by **Sport_cat** and **Dport_cat**. Together with the four original numerical attributes (**Dur**, **TotPkts**, **TotBytes**, and **SrcBytes**), they form the 12-dimensional base feature set used by both downstream models.

3.3.2 XGBoost Interaction Features

To provide richer non-linear information to the XGBoost model, six engineered interaction features are generated from the 12 base features. These features do not include any per-IP aggregation and therefore avoid introducing host identity information.

The engineered features include:

- **Bytes-per-second efficiency**

Measures the amount of transmitted data relative to the packet transmission rate.

$$F_1 = \frac{\text{ByteRate}}{\text{PktRate} + \varepsilon}$$

where ε is a small constant added to avoid division by zero.

- **Source-dominance measure**

Captures flows in which the source contributes most of the transmitted bytes over long durations.

$$F_2 = \text{SrcBytesRatio} \times \log(1 + \text{Dur})$$

- **Port-symmetry indicator**

Indicates whether source and destination ports belong to the same category.

$$F_3 = \begin{cases} 1, & \text{if Sport} = \text{Dport}, \\ 0, & \text{otherwise.} \end{cases}$$

- **Packet-density proxy**

Characterizes flows with relatively small payloads spread across many packets.

$$F_4 = \frac{\text{TotPkts}}{\text{TotBytes} + \varepsilon}$$

where ε prevents division by zero.

- **Protocol-destination port interaction**

Encodes protocol and destination-port combinations into a single interaction feature.

$$F_5 = 10 \times \text{Proto} + \text{Dport}$$

The scaling factor of 10 is chosen to separate protocol categories.

- **Byte-asymmetry magnitude**

Measures the absolute imbalance between source and destination byte contributions.

$$F_6 = |2 \times \text{SrcBytesRatio} - 1|$$

Values close to 0 indicate symmetric communication, whereas values approaching 1 correspond to highly asymmetric flows.

These six features are appended to the 12 base features, resulting in an 18-dimensional feature vector for XGBoost.

3.3.3 Temporal Features Within Window (TCN Only)

To reduce sensitivity to distribution shifts between training and validation IPs, seven additional sequence-level features are computed for each TCN sequence window:

- **win_IAT_mean**: Mean inter-arrival time within a sequence window

$$F_1 = \frac{1}{L} \sum_{i=1}^L IAT_i$$

where L denotes the sequence length.

- **win_IAT_std**: Standard deviation of inter-arrival times

$$F_2 = \sqrt{\frac{1}{L} \sum_{i=1}^L (IAT_i - \mu_{IAT})^2}$$

where

$$\mu_{IAT} = \frac{1}{L} \sum_{i=1}^L IAT_i.$$

- **win_beacon_reg**: Beaconsing regularity measure

$$F_3 = \frac{\sigma_{IAT}}{\mu_{IAT} + \varepsilon}$$

where σ_{IAT} is the inter-arrival time standard deviation, μ_{IAT} is the mean inter-arrival time, and ε is a small constant preventing division by zero.

Smaller values indicate highly periodic communication, while larger values correspond to bursty or irregular transmissions.

- **win_payload_cv**: Payload coefficient of variation

$$F_4 = \frac{\sigma_{Bytes}}{\mu_{Bytes} + \varepsilon}$$

where

$$\mu_{Bytes} = \frac{1}{L} \sum_{i=1}^L TotBytes_i$$

and

$$\sigma_{Bytes} = \sqrt{\frac{1}{L} \sum_{i=1}^L (TotBytes_i - \mu_{Bytes})^2}.$$

This feature distinguishes bursty traffic from steady transmissions.

- **flow_asymmetry**: Mean source-to-destination byte asymmetry within the window

Let

$$A_i = \frac{SrcBytes_i}{TotBytes_i + \varepsilon}$$

denote the asymmetry ratio for flow i .

The window-level statistic is

$$F_5 = \frac{1}{L} \sum_{i=1}^L A_i.$$

- **win_dst_fanout**: Destination fan-out

$$F_6 = \left| \{DstAddr_i\}_{i=1}^L \right|$$

where $|\cdot|$ denotes the number of distinct destination addresses observed within the sequence window.

- **win_dport_entropy**: Destination-port Shannon entropy

Let

$$p_j = \frac{n_j}{L}$$

be the empirical probability of observing destination port j within the window.

Then

$$F_7 = - \sum_j p_j \log(p_j + \varepsilon)$$

where ε is introduced for numerical stability.

Higher entropy values indicate communication distributed across many ports, whereas lower values correspond to concentrated communication patterns.

3.4 Data Splitting and Scaling

Prior to any scaling operation, source IPs are partitioned into training, validation, and test sets. A source IP is allowed to appear in only one split, ensuring that no traffic from the same host is present across multiple datasets.

Disjoint-set assertions are used to verify that source IP overlap does not occur, thereby preventing identity leakage between training and evaluation.

A Yeo-Johnson PowerTransformer is fitted using a random sample of 200,000 rows drawn exclusively from the training set. The transformer is applied to the 12 base features and its outputs are clipped to the range $[-4, 4]$.

The fitted scaler is serialized as `scaler.pkl` and reused without modification for validation, test, and CICIDS-2017 evaluation datasets.

The seven TCN window-level features are scaled independently using statistics computed only from training-window data to avoid information leakage.

3.5 Sequence Construction

For TCN training, flows are grouped by source IP and ordered according to `StartTime_epoch`. The ordered flows are then segmented into fixed-length windows of

$$SEQ_LEN = 20$$

using:

- `TRAIN_STEP = 10` during training
- `TEST_STEP = 1` during evaluation

Source IPs containing fewer than 20 flows are cyclically padded to reach the required sequence length.

A sequence is labeled malicious only if at least 25% of the original (pre-padding) flows are malicious.

Generated sequences are streamed directly into HDF5 caches:

- `train_seqs.h5`
- `val_seqs.h5`
- `test_seqs.h5`

using

$$SEQ_CHUNK_SIZE = 50,000$$

which ensures that only a subset of sequence tensors resides in memory at any given time.

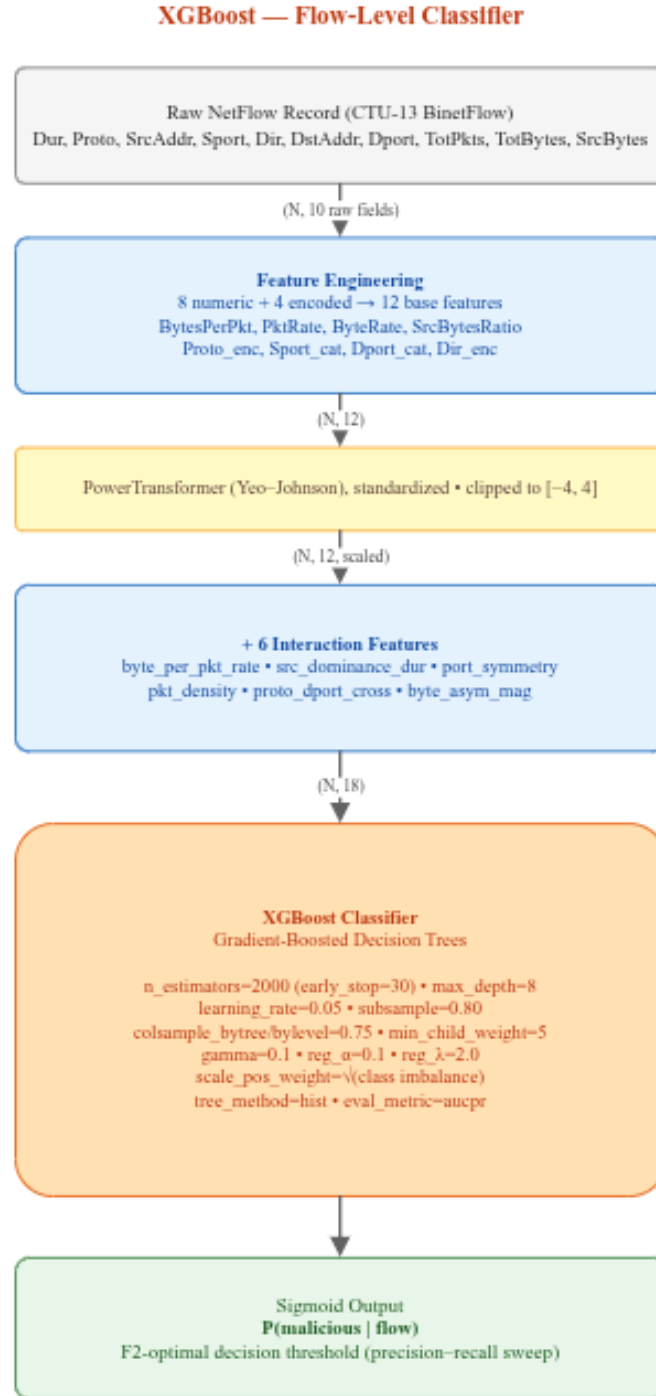


Figure 3.2: Architecture of the proposed XGBoost-based flow-level classifier for encrypted command-and-control (C2) communication detection using the CTU-13 BinetFlow dataset. Each network flow is represented by raw BinetFlow attributes, transformed through feature engineering and interaction feature generation, normalized using a Yeo-Johnson power transformation with clipping, and subsequently classified by a gradient-boosted decision tree ensemble. The model outputs the probability that an individual network flow corresponds to malicious C2 activity.

3.5.1 XGBoost (Per-Flow Classifier)

The XGBoost classifier uses the `gpu_hist` tree construction algorithm.

The model operates on:

- 12 base features
- 6 engineered interaction features

for a total of 18 input dimensions.

Class imbalance is addressed through the `scale_pos_weight` parameter without requiring data resampling, thereby preserving the natural distribution of network traffic.

3.5.2 Temporal Convolutional Network (Sequence Classifier)

The TCN operates independently of XGBoost and consumes sequence tensors of shape:

$$(20, 19)$$

The architecture consists of:

1. TCN backbone:

- 64 filters
- Kernel size = 3
- Dilation rates = [1, 2, 4, 8]
- ELU activation
- Dropout = 10%

The resulting receptive field spans 30 timesteps, exceeding the 20-timestep input length.

2. Global temporal aggregation through addition across timesteps.

3. Dense classification head:

- Dense(64) + ELU + L2 regularization
- Dropout(20%)
- Dense(32) + ELU + L2 regularization
- Dropout(10%)

4. Sigmoid output neuron with bias initialized to the log-odds of the malicious fraction in the training data.

Initializing the output bias to the class prior helps the network begin training closer to the expected probability distribution rather than starting from an unbiased estimate.

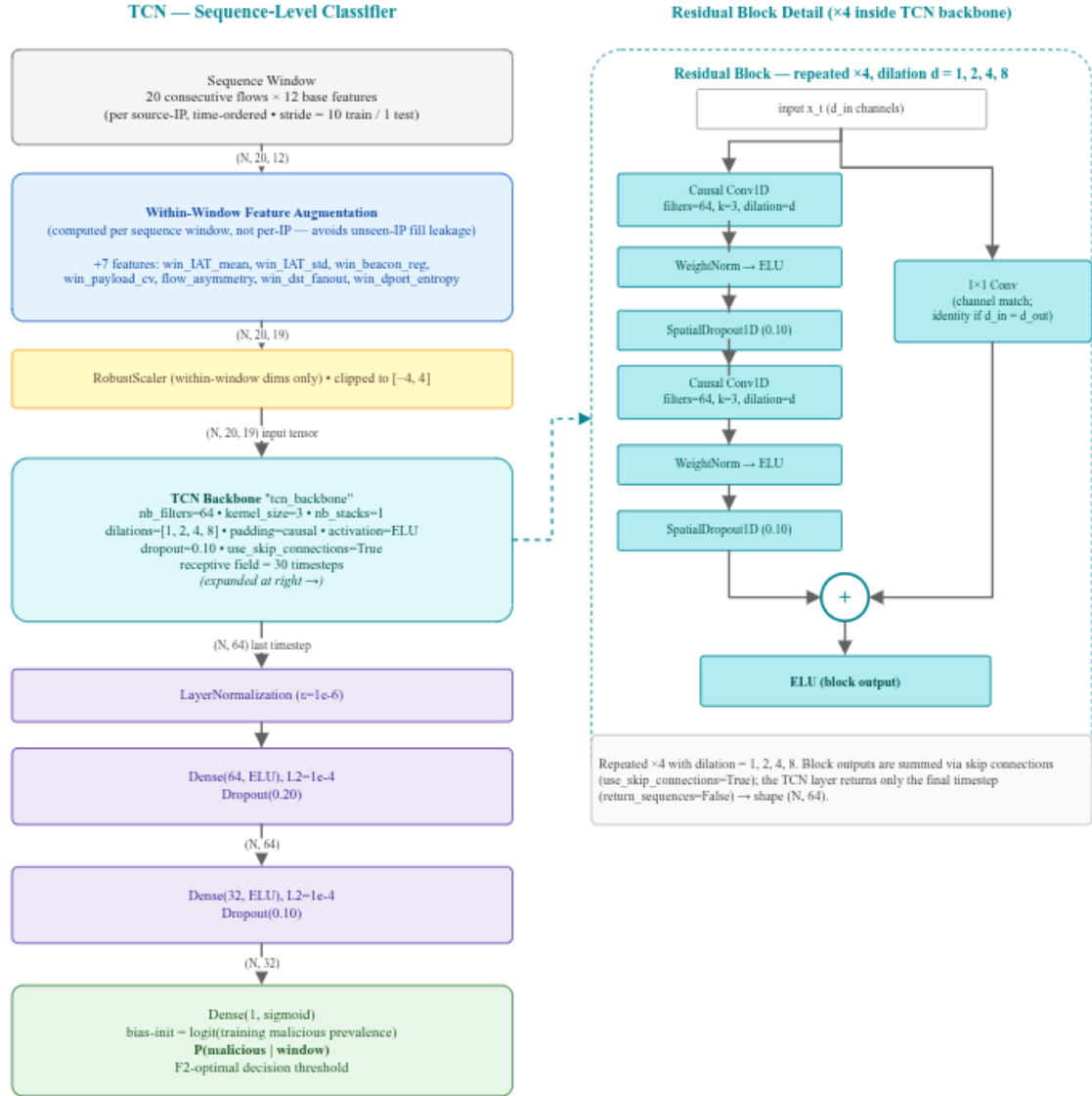


Figure 3.3: Architecture of the proposed Temporal Convolutional Network (TCN) sequence classifier for encrypted command-and-control (C2) traffic detection using the CTU-13 BinetFlow dataset. The model processes windows of 20 consecutive flows augmented with within-window temporal features, followed by robust scaling, a TCN backbone composed of dilated residual blocks, and fully connected layers that estimate the probability of a malicious communication sequence. The right panel illustrates the residual block structure employed within the TCN backbone.

3.6 Training Configuration

The TCN is trained using Binary Focal Crossentropy loss, which reduces the influence of easily classified benign sequences and concentrates learning on difficult minority-class examples.

Training data is streamed directly from the HDF5 cache without loading the entire dataset into memory.

The training configuration is:

- Batch size = 512
- Maximum epochs = 60
- Learning rate = 5×10^{-4}
- Early stopping patience = 10 epochs

The XGBoost and TCN models remain completely independent throughout training, validation, and evaluation. No ensembling, score fusion, stacking, or averaging is performed. Consequently, each model's probability outputs are analyzed separately in Chapter 5 in order to compare the behavior of per-flow statistical learning and per-sequence temporal learning independently.

Chapter 4

IMPLEMENTATION

4.1 Software Environment and Libraries

The system is implemented in Python. TensorFlow/Keras is used for the Temporal Convolutional Network (TCN), XGBoost (GPU-accelerated) is used for the per-flow classifier, and Scikit-Learn is used for preprocessing and calibration tasks including PowerTransformer scaling, label encoding, and logistic/isotonic recalibration.

The `keras-tcn` package is used to implement the dilated causal convolution layers required for sequence modeling.

Model training is performed on an NVIDIA GTX 1650 GPU with 4 GB of VRAM. At the beginning of each training session, GPU memory usage is explicitly limited to 3 GB in order to reserve resources for the operating system and background processes. Additionally, the Keras backend data type is configured as `float32` to ensure predictable numerical behavior throughout training and inference.

4.2 Memory-Bounded Pipeline Design

To enable training on the complete multi-scenario CTU-13 dataset within limited hardware resources, three memory-efficient engineering strategies are employed:

1. **Chunked CSV Loading**

BinetFlow files are loaded using fixed-size chunks of 500,000 rows, as described in Section 3.2, instead of loading the complete dataset into memory.

2. **HDF5 Sequence Caching**

After feature engineering and scaling, TCN sequences are generated and written directly to disk using HDF5 storage. Sequences are flushed in batches of 50,000 windows through the `build_sequences_to_hdf5` pipeline. As a result, the complete sequence tensor is never stored entirely in RAM.

3. **tf.data Streaming from HDF5**

During training, a custom generator (`hdf5_tf_dataset_indexed`) streams data directly from the HDF5 cache. Sample indices are shuffled in memory, sorted temporarily for efficient contiguous disk access, and then restored to their original randomized order before being passed to the model. This preserves stochastic gradient descent behavior while improving disk I/O efficiency.

Together, these techniques enable processing and training on the entire CTU-13 corpus without requiring the complete dataset to reside in memory at any stage of the pipeline.

4.3 Training-Time Diagnostics

Keras provides an internal approximation of the Area Under the ROC Curve (AUC), but this approximation can become inaccurate when model predictions are concentrated within a narrow probability range.

To obtain reliable validation metrics, a custom callback named `RealValMetrics` is implemented. At the end of each epoch, the callback computes:

- ROC-AUC using `roc_auc_score`
- PR-AUC using `average_precision_score`

from Scikit-Learn using the raw validation predictions.

The callback also monitors prediction collapse by measuring the standard deviation of output probabilities. If the standard deviation approaches zero, the model is flagged as producing nearly constant predictions, preventing misleading interpretations of validation performance.

4.4 CICIDS-2017 Cross-Dataset Inference Pipeline

To evaluate generalization performance on a structurally different network environment, a dedicated inference pipeline is implemented for the CICIDS-2017 dataset.

The pipeline loads and concatenates all eight day-wise CICIDS-2017 CSV files and converts their schema into a format compatible with CTU-13 BinetFlow records. The same feature engineering procedure used during training is then applied to generate:

- TotPkts
- TotBytes
- SrcBytes
- Dir

along with all derived statistical features.

Using the same feature engineering pipeline ensures that CICIDS-2017 traffic is represented in the same feature space as CTU-13 traffic.

The previously fitted CTU-13 PowerTransformer is reused during inference. However, significant distribution shifts were observed in several byte-related features, particularly:

- TotBytes
- SrcBytes
- BytesPerPkt

Therefore, only the Yeo–Johnson transformation parameters associated with these features are re-estimated using CICIDS-2017 data, while the remaining transformation parameters remain unchanged.

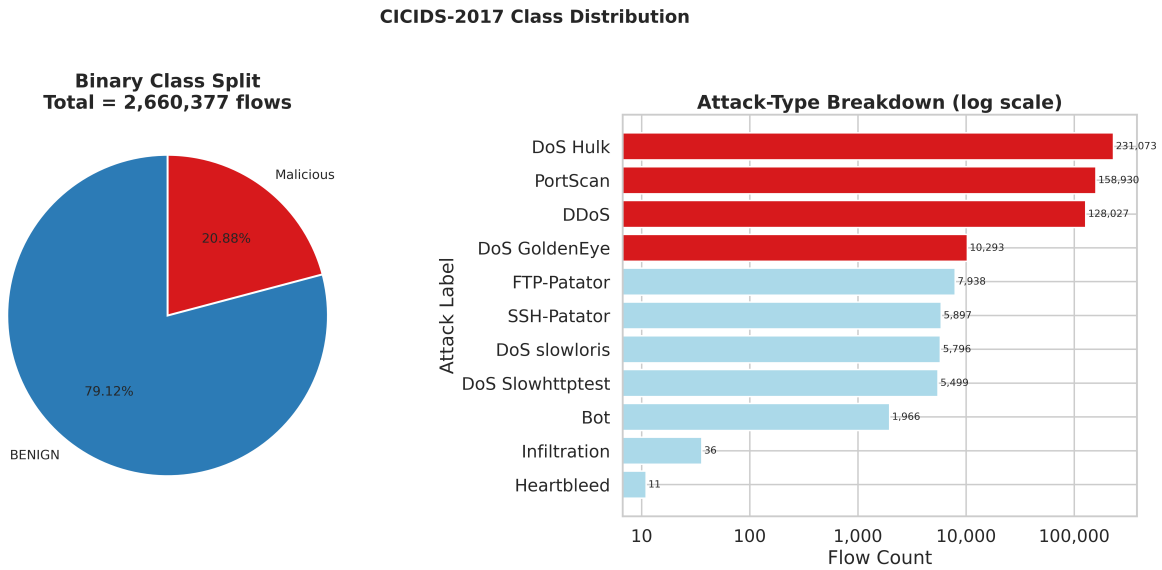


Figure 4.1: Class distribution of the CICIDS-2017 dataset used for cross-dataset evaluation. The left panel shows the binary class split, where benign traffic constitutes 79.12% of all flows and malicious traffic constitutes 20.88%. The right panel presents the distribution of attack categories on a logarithmic scale. DoS Hulk, PortScan, and DDoS attacks dominate the malicious traffic, while several attack categories such as Infiltration and Heartbleed are comparatively rare. The diversity of attack types and their distributional differences from CTU-13 contribute to the domain shift observed during cross-dataset evaluation.

4.4.1 XGBoost Domain Recalibration

When directly evaluated on CICIDS-2017, the CTU-13-trained XGBoost model produces only weak discrimination performance (ROC-AUC approximately 0.58), indicating substantial domain shift.

To improve performance, a domain recalibration pipeline is applied:

1. Compute per-feature separability weights using the squared deviation of each feature’s individual ROC-AUC from 0.5.
2. Construct a recalibration feature matrix consisting of:
 - Raw XGBoost probability scores
 - Reweighted feature values
3. Train a logistic regression recalibrator using a 30% hold-in split.
4. Apply isotonic regression to improve probability calibration.
5. Select an operating threshold using the F_2 score criterion, prioritizing recall over precision for security monitoring applications.

4.4.2 TCN Domain Recalibration

For TCN evaluation, the same sequence construction procedure used during training is repeated on CICIDS-2017 traffic. This includes:

- Grouping flows by source IP
- Cyclic padding of short flow histories
- Generation of identical within-window temporal statistics

The TCN performs inference in CPU-only mode.

To recalibrate predictions, the following information is combined:

- Raw TCN output probabilities
- Mean-aggregated window-level features

The resulting feature matrix is then processed through the same logistic regression and isotonic regression recalibration pipeline used for XGBoost.

4.5 Independent Output Reporting

As part of the design decision to keep the statistical and temporal models completely independent, no ensemble, score fusion, stacking, or averaging is performed during evaluation.

The CICIDS-2017 inference pipeline generates separate result files:

- `xgb_results.csv`
- `tcn_results.csv`

In addition, a per-source-IP alert summary file is generated. For each source IP, the report contains:

- Maximum XGBoost probability
- XGBoost alert count
- Maximum TCN probability
- TCN alert count

These results are reported side-by-side rather than being combined into a single score. This design enables direct comparison of the detection behavior of the statistical and temporal models and facilitates the analysis presented in Chapter 5.

4.6 Live Deployment System: NIDS-Live

Alongside the offline training and evaluation pipeline described in Chapters 3 and 4, both trained models were integrated into a standalone real-time intrusion detection platform named *NIDS-Live*. The objective of this deployment system is to demonstrate that the trained XGBoost and TCN models can operate on live network traffic without modification to their underlying inference procedures.

4.6.1 Architecture

NIDS-Live is implemented as a Flask-based application organized into three major layers:

1. Capture Layer
2. Inference Pipeline Layer
3. Dashboard Layer

The capture layer (**capture/**) supports two packet capture engines that are automatically selected depending on the operating system and network interface configuration:

- An NfStream-based capture engine for high-throughput packet collection on Linux and macOS.
- A Scapy-based packet capture engine with a custom flow-table implementation for Windows compatibility and fallback operation.

This dual-engine architecture enables deployment across multiple operating systems without relying on a single platform-specific capture dependency.

The inference pipeline (**pipeline/**) consists of a sequence of modular processing stages including feature engineering, scaling, XGBoost inference, TCN inference, and domain adaptation. Feature engineering performs the same ratio calculations and port categorization procedures used during model training. The scaling stage applies the Power-Transformer and feature clipping operations. Separate inference modules execute the XGBoost and TCN models independently.

A domain adaptation layer performs byte-feature scaling adaptation and isotonic recalibration to compensate for distribution drift in live traffic environments. Flow records are processed through a queue-based orchestration mechanism, while a shared-state management module ensures thread-safe communication between background packet capture processes and Flask request handlers.

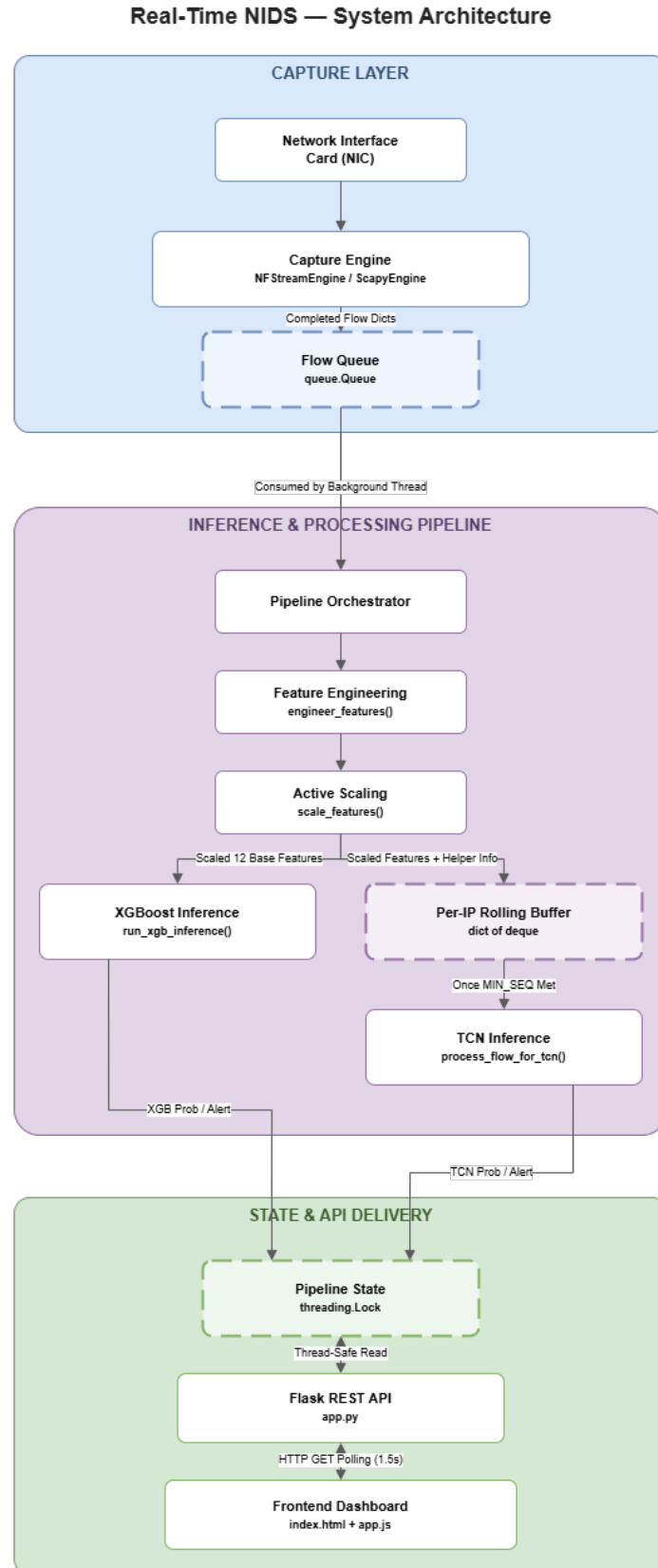


Figure 4.2: Architecture of the NIDS-Live deployment system. The capture layer acquires network traffic through NFStream or Scapy-based packet capture engines and stores completed flows in a queue. The inference pipeline performs feature engineering, scaling, and independent XGBoost and TCN inference. Results are maintained within a shared pipeline state and exposed through a Flask REST API to a real-time web dashboard.

4.6.2 Inference Parity

A key design objective of NIDS-Live is the elimination of train–serve skew. Every transformation applied to live traffic is mathematically identical to the transformations used during offline training and CICIDS-2017 evaluation.

The following serialized artifacts are loaded directly by the deployment system:

- `scaler.pkl`
- `win_scaler.pkl`
- `clip_bounds.pkl`
- `le_proto.pkl`
- `le_dir.pkl`

As a result, categorical encoding, interaction feature generation, Yeo–Johnson transformation, clipping, and sequence construction remain fully consistent between offline evaluation and live deployment.

4.6.3 TCN Sequence Windowing in a Streaming Environment

The TCN requires fixed-length sequences of twenty flows per source IP address. To support sequence-based inference in a streaming environment, NIDS-Live maintains an IP-indexed rolling queue for each active source address.

When fewer than twenty flows are available for a given source IP, the same cyclic padding strategy used during offline sequence construction is applied. This allows TCN-based detection to begin immediately rather than waiting for a complete flow history to accumulate.

The streaming implementation therefore preserves consistency with the offline training procedure while minimizing detection latency for newly observed hosts.

4.6.4 Independent Model Operation

Consistent with the design philosophy adopted throughout this project, the XGBoost and TCN models operate independently within the deployment system.

Each model maintains:

- Separate inference paths
- Independent classification thresholds
- Individual threshold auditing records
- Dedicated alert streams

Threshold values are recorded in `threshold_audit.jsonl`, enabling complete auditability of operational changes. No score fusion, stacking, averaging, or ensemble decision-making is performed within the deployment pipeline.

4.6.5 Operator Dashboard

NIDS-Live provides a web-based operator dashboard implemented using Flask templates and client-side JavaScript.

The dashboard enables:

- Real-time flow monitoring
- Security alert visualization
- Independent threshold adjustment for XGBoost and TCN
- Historical threshold audit review
- Live inspection of active flow tables

Configuration changes can be applied dynamically without restarting the application, allowing operators to adapt detection sensitivity in response to changing network conditions.

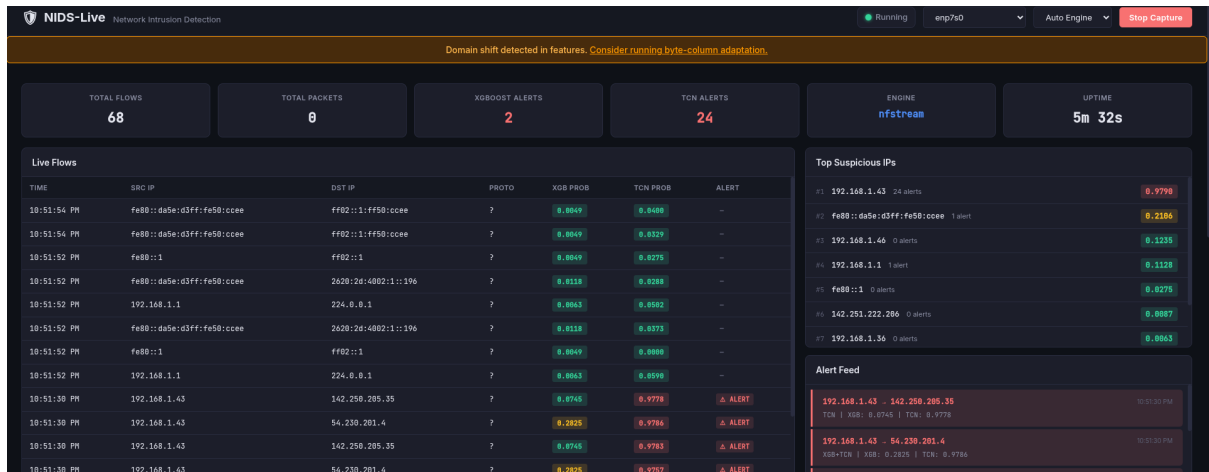


Figure 4.3: NIDS-Live operator dashboard during live network monitoring. The interface displays real-time flow statistics, model-specific alert counts, suspicious source IP rankings, live flow records, and an alert feed generated independently by the XGBoost and TCN detection pipelines. The dashboard also provides visibility into domain-shift warnings and capture engine status.

Chapter 5

RESULTS AND DISCUSSION

5.1 Experimental Setup

After chunked loading and preprocessing across all thirteen CTU-13 scenarios, the dataset contained a total of 19,976,700 labeled flows, consisting of 19,532,001 benign flows (97.8%) and 444,699 malicious flows (2.2%).

Following the source-IP-disjoint split procedure described in Chapter 3, 12,788,415 flows were allocated to the training set, 1,243,546 flows to the validation set, and 5,944,739 flows to the test set. No source IP address appeared in more than one split, thereby preventing identity leakage between training and evaluation.

All results reported in this chapter are obtained from the XGBoost and TCN models independently. No ensembling, score fusion, stacking, or averaging is performed.

5.1.1 XGBoost (Per-Flow) Results on CTU-13

The XGBoost classifier was trained using an 18-dimensional feature set comprising 12 base traffic features and six engineered interaction terms. Evaluation on the held-out CTU-13 test set containing 5,917,536 flows, including 6,801 malicious flows, yielded a ROC-AUC of 0.9952 and a PR-AUC of 0.6645.

An operating threshold of 0.8588 was selected using an F2-optimal precision–recall sweep, resulting in a malicious-class precision of 0.41, recall of 0.72, and an F1-score of 0.5223. The corresponding confusion matrix and probability distributions are presented below.

The high ROC-AUC demonstrates strong separation between benign and malicious traffic in the probability space. Gain analysis further showed that the engineered protocol–destination-port interaction feature (`proto_dport_cross`) achieved the highest importance score (0.235), exceeding all base features. This finding validates the feature engineering strategy proposed in Section 3.3.2 and suggests that protocol–port combinations capture behavioural information unavailable from either feature individually.

The disparity between the high ROC-AUC and the threshold-dependent F1-score is primarily attributable to the severe class imbalance in the evaluation set, where malicious flows constitute only approximately 0.11% of all traffic. At the selected threshold, the model correctly identified 4,896 malicious flows while missing 1,905, producing 7,045 false positives and yielding a precision of 0.41. Consequently, PR-AUC and F2-score provide more operationally meaningful performance measures for NIDS deployment under highly imbalanced traffic conditions.

The selected threshold intentionally favours recall while maintaining an acceptable false-positive burden for practical deployment within the NIDS-Live dashboard. Lowering the threshold would recover additional malicious flows but at the expense of substantially increasing false alarms.

Despite its excellent ranking capability, the XGBoost model performs classification independently for each flow and therefore lacks awareness of temporal behavioural patterns. Periodic low-volume beaconing activity, characteristic of command-and-control communication, may appear statistically benign when individual flows are considered in isolation. This limitation motivates the use of a Temporal Convolutional Network (TCN), which incorporates temporal context and achieves a substantially higher malicious-class F1-score of 0.7364, demonstrating the additional detection value provided by sequence modelling.

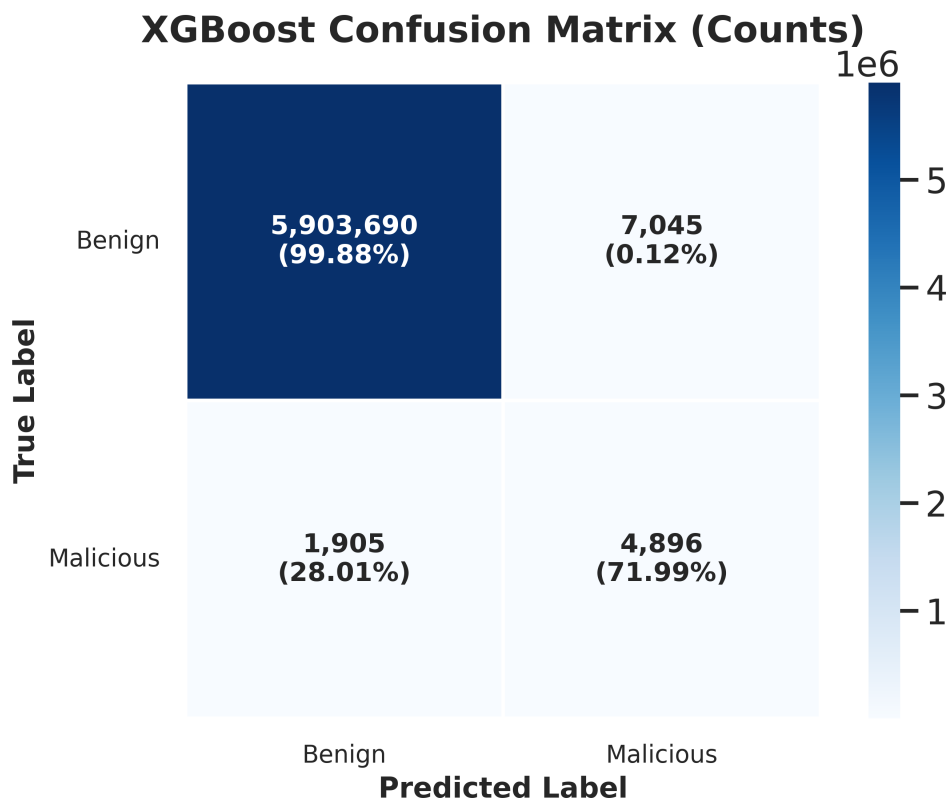


Figure 5.1: Confusion Matrix of the XGBoost classifier on the CTU-13 test dataset. The model correctly classified 5,903,690 benign flows and 4,896 malicious flows, while producing 7,045 false positives and 1,905 false negatives. The results correspond to a malicious-class recall of 71.99% at the selected operating threshold.

5.1.2 TCN (Sequence-Level) Results on CTU-13

The Temporal Convolutional Network (TCN) was trained on 1,088,104 sequences, of which 20,090 (1.85%) were malicious. Evaluation was performed on a source-IP-disjoint test set containing 39,971 sequences, including 738 malicious sequences. The model achieved a ROC-AUC of 0.9495 and a PR-AUC of 0.8140.

An operating threshold of 0.0253 was selected using an F2-optimal precision–recall

sweep to prioritise malicious traffic recall. At this threshold, the TCN attained a malicious-class precision of 0.65, recall of 0.84, and an F1-score of 0.7364.

The TCN correctly classified 38,900 benign sequences and 619 malicious sequences, producing 333 false positives and 119 false negatives. Compared with the XGBoost model presented in Section 5.2, the TCN improved malicious-class precision, recall, and F1-score by 0.24, 0.12, and 0.214, respectively. These improvements were achieved despite operating at the sequence level, where each evaluation sample comprises 20 consecutive flows from a source IP address.

The superior performance of the TCN can be attributed to its access to seven temporal descriptors within each sequence window. Features such as beaconing regularity (`win_beacon`), destination fan-out (`win_dst_fanout`), and destination-port entropy capture behavioural characteristics of command-and-control communication that remain invisible to per-flow classifiers. Although individual beaconing flows may exhibit benign packet, byte, and duration statistics, their periodicity, limited destination diversity, and stable port usage produce distinctive temporal signatures when observed over multiple consecutive flows.

Despite achieving a lower ROC-AUC than XGBoost, the TCN demonstrates superior detection performance at the deployment-relevant operating threshold. ROC-AUC evaluates ranking quality across all thresholds, whereas F1-score and recall better reflect practical malicious traffic detection capability. The sequence-level label assignment strategy and the presence of mixed benign-malicious windows introduce label noise that can slightly reduce ranking smoothness, thereby affecting ROC-AUC. Nevertheless, the TCN maintains substantially higher recall and F1-score at the F2-optimal threshold.

These findings directly validate the central hypothesis of this work: malicious C2 traffic often appears statistically benign when individual flows are analysed in isolation, whereas temporal sequence modelling enables the detection of behavioural indicators such as periodic beaconing, low destination fan-out, and consistent port usage. The observed improvements of 0.214 in F1-score and 0.12 in recall quantify the detection benefit provided by temporal context and demonstrate its importance for encrypted C2 detection in realistic network environments.

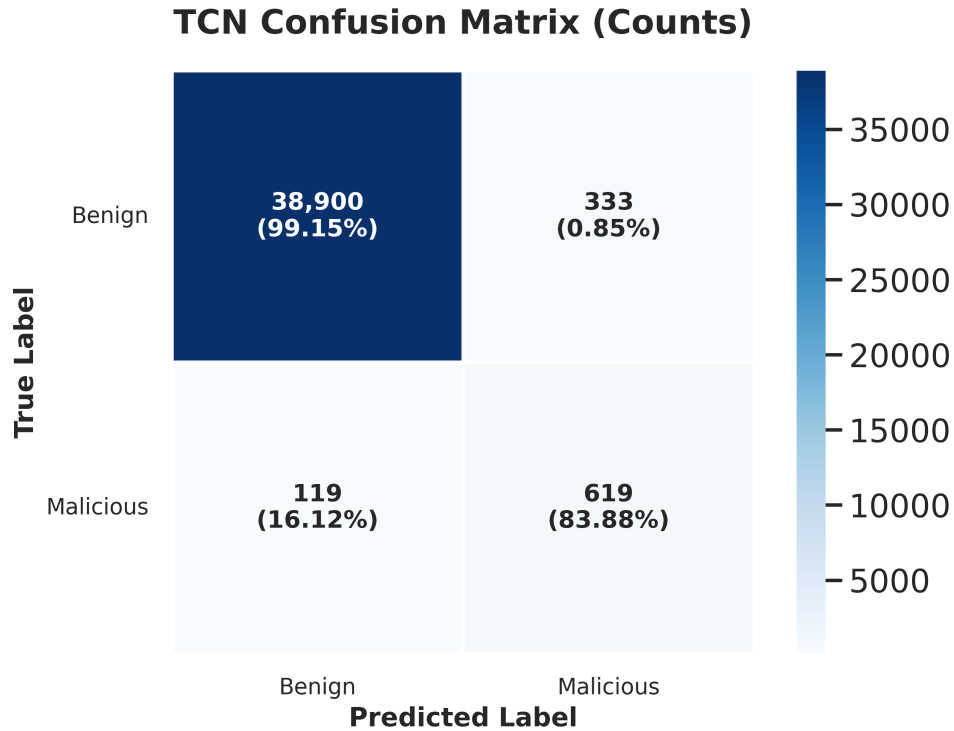


Figure 5.2: Confusion Matrix of the Temporal Convolutional Network (TCN) on the CTU-13 sequence-level test set. The model correctly classified 38,900 benign sequences and 619 malicious sequences, while producing 333 false positives and 119 false negatives. The resulting malicious-class recall was 83.88%, demonstrating the TCN’s ability to capture temporal behavioural patterns associated with botnet activity.

5.2 Cross-Dataset Generalization on CICIDS-2017

5.2.1 Base (Non-Recalibrated) Performance

Both models experienced substantial performance degradation when directly transferred from CTU-13 to the structurally different CICIDS-2017 dataset. The XGBoost model achieved a ROC-AUC of 0.5798 and a PR-AUC of 0.2559, with a benign-to-malicious probability separation ratio of only $0.93\times$, indicating limited retained discriminative capability. The TCN exhibited an even larger performance decline, obtaining a ROC-AUC of 0.5027 and a PR-AUC of 0.2042 on 2,539,717 CICIDS-2017 sequences, corresponding to near-random classification performance.

This degradation does not imply that the models learned meaningless representations. Rather, it highlights a fundamental challenge in NIDS research: network traffic statistics are highly dependent on network topology, attack characteristics, and data collection methodologies. CTU-13 consists primarily of botnet-generated C2 traffic collected in a university environment, whereas CICIDS-2017 was generated within a simulated enterprise network containing substantially different attack scenarios. Consequently, byte-magnitude and flow-rate features occupy different regions of the feature space, causing decision boundaries learned on CTU-13 to lose their discriminative capability when applied directly to CICIDS-2017.

The TCN suffers a more pronounced performance drop than XGBoost because do-

main shifts affect not only the base flow features but also the derived temporal descriptors, including inter-arrival statistics, beaconing regularity, and payload variability. Since the TCN learns decision boundaries in a higher-dimensional space incorporating both flow-level and temporal features, changes in the underlying traffic distributions cause its predicted probabilities to approach the class prior distribution. In contrast, XGBoost retains limited residual discriminative capability through comparatively transferable features such as `Proto_enc` and `SrcBytesRatio`.

Although disappointing from an operational perspective, these near-random baseline results are scientifically informative. They demonstrate that both models learned domain-specific representations from CTU-13 and provide strong motivation for the recalibration framework proposed in Section 4.4. Furthermore, the observed performance degradation is independent of identity leakage, which was mitigated through source-IP-disjoint splitting. Instead, the degradation reflects genuine distribution shifts between independently collected datasets, emphasising the need for adaptation mechanisms to enable practical deployment across diverse network environments.

“`latex`

5.2.2 Recalibrated Performance

After applying the domain recalibration pipeline described in Section 4.4, both models exhibited substantial performance improvements. The recalibration procedure consisted of feature-separability reweighting followed by logistic and isotonic calibration using a 30% hold-in subset of CICIDS-2017. Importantly, these gains were achieved without retraining either model.

Table 5.1: Performance Before and After Domain Recalibration

Model	Metric	Base	Recalibrated
XGBoost	ROC-AUC	0.5798	0.9186
XGBoost	PR-AUC	0.2559	0.6663
TCN	ROC-AUC	0.5027	0.9964
TCN	PR-AUC	0.2042	0.9803

The recalibrated TCN outperformed XGBoost across all evaluation metrics, achieving a ROC-AUC of 0.9964, PR-AUC of 0.9803, F1-score of 0.9382, and F₂-score of 0.9689, compared to 0.9186, 0.6663, 0.7052, and 0.8152 for XGBoost, respectively. Relative to their unrecalibrated baselines, the ROC-AUC increased by 0.3388 for XGBoost and by 0.4937 for the TCN, demonstrating that the primary limitation of cross-dataset deployment was probability miscalibration and feature-scale mismatch rather than an absence of transferable representations.

Feature transferability analysis helps explain the stronger recovery observed for the TCN. The most transferable TCN features were `win_dst_fanout` (AUC = 0.8522), `win_payload_cv` (AUC = 0.8435), and `win_fasym_mean` (AUC = 0.7916), all of which describe behavioural characteristics that are largely independent of network topology and traffic volume. In comparison, the most transferable XGBoost features, namely `SrcBytesRatio` (AUC = 0.7377), `Proto_enc` (AUC = 0.7176), and `proto_dport_cross` (AUC = 0.7068), exhibited lower separability on CICIDS-2017, limiting the extent of performance recovery.

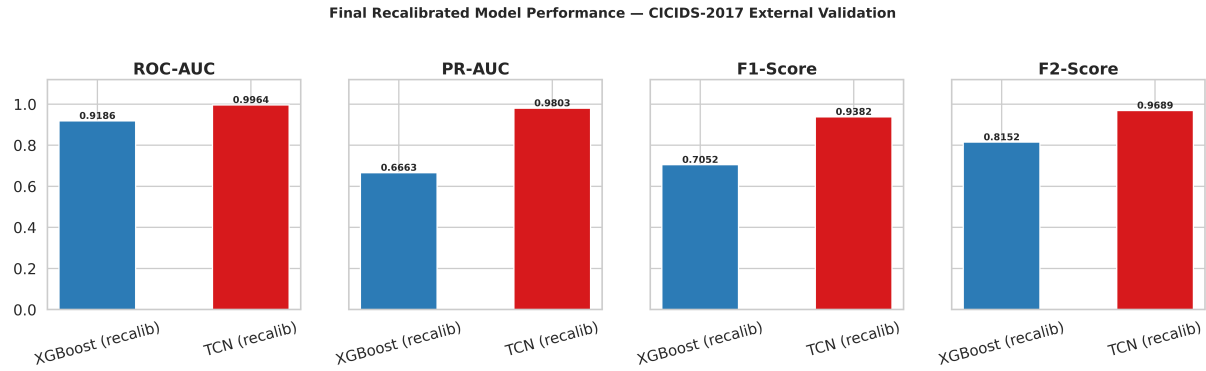


Figure 5.3: Performance comparison of the recalibrated XGBoost and TCN models on the CICIDS-2017 dataset. The TCN consistently outperformed XGBoost across ROC-AUC, PR-AUC, F1-score, and F₂-score, highlighting the benefit of temporal sequence modelling under domain adaptation.

The large post-recalibration PR-AUC gap (0.9803 for TCN versus 0.6663 for XGBoost) is particularly significant, as PR-AUC better reflects operational performance under severe class imbalance. The superior precision–recall trade-off achieved by the TCN reinforces the findings of Sections 5.2 and 5.3 that temporal sequence modelling provides greater practical detection capability than per-flow statistical classification.

Overall, these results suggest an important design principle for future cross-domain NIDS: structural and behavioural communication features, such as destination fan-out, payload variability, flow asymmetry, and protocol–port interactions, generalise substantially better across network environments than magnitude-based statistics including total bytes, packet rate, and flow duration. “

5.3 Ablation Study

5.3.1 Interaction Feature Ablation via XGBoost Gain Importance

The XGBoost classifier was trained using 18 features comprising 12 base flow statistics and six engineered interaction features (F_1 – F_6). Gain importance provides an implicit post-hoc ablation measure by quantifying each feature’s contribution to reducing the ensemble training loss. Features with negligible gain are likely to have limited influence on model performance and may be considered for removal. Table 5.2 summarises the gain importance values and corresponding ablation recommendations.

The gain distribution is highly concentrated, with the engineered protocol–destination–port interaction feature (`proto_dport_cross`) contributing the largest gain (0.235), exceeding all base features. This suggests that protocol and destination–port combinations encode discriminative behaviour not captured by individual features alone. In contrast, `port_symmetry`, `byte_asym_mag`, and `TotPkts` exhibit minimal gain and are potential candidates for removal.

Interestingly, features such as `ByteRate` and `PktRate`, despite showing substantial class separation, contribute less gain than categorical variables including `Proto_enc` and `Sport_cat`. This indicates that protocol and port-related attributes provide more stable decision boundaries under varying traffic conditions.

Table 5.2: XGBoost gain importance and ablation recommendations.

Feature	Category	Gain	Verdict
proto_dport_cross (F_5)	Interaction	0.235	Retain
Proto_enc	Base	0.200	Retain
Sport_cat	Base	0.183	Retain
Dport_cat	Base	0.092	Retain
BytesPerPkt	Base	0.053	Retain
ByteRate	Base	0.038	Retain
PktRate	Base	0.033	Retain
Dur	Base	0.029	Retain
SrcBytesRatio	Base	0.021	Retain
pkt_density (F_4)	Interaction	0.016	Retain
TotBytes	Base	0.015	Retain
Dir_enc	Base	0.015	Retain
SrcBytes	Base	0.013	Retain
src_dominance_dur (F_2)	Interaction	0.012	Retain
byte_per_pkt_rate (F_1)	Interaction	0.010	Retain
byte_asym_mag (F_6)	Interaction	0.006	Candidate removal
TotPkts	Base	0.004	Candidate removal
port_symmetry (F_3)	Interaction	≈ 0.000	Remove

Overall, the results suggest that a reduced feature set excluding `port_symmetry`, `byte_asym_mag`, and `TotPkts` could preserve most of the model’s discriminative capability while lowering computational overhead.

5.3.2 Temporal Context Ablation: XGBoost as the Ablated Baseline

A direct comparison between the XGBoost and TCN models provides a natural temporal context ablation. Both models utilise the same 12-dimensional base flow features; however, XGBoost classifies flows independently, whereas the TCN processes windows of 20 consecutive flows augmented with seven temporal descriptors, including inter-arrival statistics, beaconing regularity, payload variability, flow asymmetry, destination fan-out, and destination-port entropy. Consequently, the XGBoost model serves as an ablated baseline in which temporal context is removed while preserving the underlying flow statistics.

Table 5.3 summarises the performance of both models on the CTU-13 test set. Precision–recall metrics are emphasised, as they better reflect malicious traffic detection under class imbalance.

The inclusion of temporal context improves PR-AUC by 0.150 and F1-score by 0.214, demonstrating the benefit of sequence modelling for identifying behavioural indicators such as beaconing activity and destination fan-out. Although XGBoost attains a higher ROC-AUC, this advantage does not translate into better performance at operational thresholds. At the F2-optimal threshold, the TCN achieves a malicious-class F1-score of 0.74 and recall of 0.84, compared with 0.52 and 0.72 for XGBoost.

Table 5.3: Temporal context ablation. XGBoost acts as the ablated baseline for the TCN.
[†] F2-scores are from the CICIDS-2017 recalibrated evaluation.

Model	ROC-AUC	PR-AUC	F1	F2*
XGBoost (per-flow)	0.9952	0.6645	0.5223	0.8152 [†]
TCN (sequence)	0.9495	0.8140	0.7364	0.9689 [†]
Δ (TCN – XGB)	−0.046	+0.150	+0.214	+0.154

These findings suggest that encrypted C2 traffic often exhibits malicious characteristics only when observed over time. Individual flows may appear benign in isolation, whereas temporal patterns such as periodic communication and consistent destination behaviour provide additional discriminative information that sequence models can effectively exploit.

5.3.3 Cross-Domain Feature Transferability Analysis

The CICIDS-2017 cross-dataset evaluation provides an opportunity to assess feature transferability by examining the univariate ROC-AUC of individual features. Features with high AUC values are expected to retain discriminative capability across network environments, whereas features with near-random performance primarily contribute to in-distribution detection.

For the XGBoost feature set, `SrcBytesRatio` (AUC = 0.738), `Proto_enc` (AUC = 0.718), and `proto_dport_cross` (AUC = 0.707) exhibit the strongest transferability, suggesting that structural characteristics such as flow asymmetry and protocol-port combinations generalise well beyond the CTU-13 environment. In contrast, `Dir_enc` and `port_symmetry` achieve AUC values close to 0.5, indicating negligible cross-domain discriminative capability.

Table 5.4 summarises the transferability of TCN window-level features, grouped into three tiers based on their univariate ROC-AUC values.

Five features achieve Tier 1 transferability (AUC $\geq$ 0.79), namely `win_dst_fanout`, `Proto_enc`, `win_pay_cv`, `SrcBytesRatio`, and `win_fasym_mean`. These features capture structural and behavioural characteristics that remain discriminative across datasets, with destination fan-out emerging as the strongest transferable sequence-level signal.

Conversely, rate- and magnitude-based features such as `PktRate`, `Dur`, `TotPkts`, and `TotBytes` demonstrate poor transferability, suggesting that their effectiveness is highly dependent on dataset-specific traffic characteristics. These findings indicate that a transfer-optimised TCN could potentially retain only Tier 1 features while preserving much of its cross-domain discriminative capability.

Table 5.4: TCN window-level feature transferability ranked by univariate ROC-AUC on CICIDS-2017.

Feature	AUC	Transferability
win_dst_fanout	0.852	Tier 1 – Transferable
Proto_enc	0.847	Tier 1 – Transferable
win_pay_cv	0.844	Tier 1 – Transferable
SrcBytesRatio	0.801	Tier 1 – Transferable
win_fasym_mean	0.792	Tier 1 – Transferable
Dport_cat	0.740	Tier 2 – Partial
win_iat_mean	0.689	Tier 2 – Partial
win_iat_std	0.689	Tier 2 – Partial
win_beacon	0.688	Tier 2 – Partial
Sport_cat	0.678	Tier 2 – Partial
SrcBytes	0.669	Tier 2 – Partial
win_dport_ent	0.639	Tier 2 – Partial
ByteRate	0.605	Tier 2 – Partial
PktRate	0.555	Tier 3 – Non-transferable
Dur	0.540	Tier 3 – Non-transferable
BytesPerPkt	0.519	Tier 3 – Non-transferable
TotPkts	0.519	Tier 3 – Non-transferable
TotBytes	0.502	Tier 3 – Non-transferable
Dir_enc	0.500	Tier 3 – Non-transferable

5.4 Summary of Findings

The principal findings of this study are summarized below.

1. Both models achieved strong in-domain performance on CTU-13.

XGBoost demonstrated excellent flow-level ranking capability, achieving a ROC-AUC of 0.9952. The engineered `proto_dport_cross` feature contributed the highest gain (0.235), validating the interaction-based feature engineering strategy. The TCN attained a ROC-AUC of 0.9495, with its lower ranking performance attributable to sequence-level evaluation and noisier label boundaries.

2. Temporal sequence modelling substantially improved operational detection performance.

The TCN achieved a malicious-class F1-score of 0.7364 and recall of 0.84, outperforming XGBoost, which obtained an F1-score of 0.5223 and recall of 0.72. These results confirm the central hypothesis of this work that command-and-control beaconing generates temporal signatures, including periodic inter-arrival times, low destination fan-out, and stable port usage, that remain hidden from per-flow classifiers.

3. Direct cross-dataset transfer resulted in near-random performance.

Without adaptation, XGBoost and TCN achieved ROC-AUC scores of only 0.5798 and 0.5027, respectively, on CICIDS-2017. This demonstrates that byte-magnitude and flow-rate statistics are highly dependent on the network environment and do not readily generalise across datasets with different traffic characteristics.

4. Lightweight domain recalibration effectively restored cross-dataset performance.

Feature-separability reweighting combined with logistic or isotonic calibration increased ROC-AUC to 0.9186 for XGBoost and 0.9964 for the TCN without re-training either model. The stronger recovery observed for the TCN suggests that temporal and structural representations possess greater transferability across network environments.

5. Structural and behavioural features generalise better than magnitude-based features.

Features such as destination fan-out, payload variability, flow asymmetry, and protocol–port interactions exhibited substantially higher transferability than magnitude-dependent statistics including total bytes, packet counts, and flow duration. These findings suggest that future cross-domain NIDS should prioritise behavioural and temporal representations to minimise recalibration effort while maintaining robust detection capability.

Chapter 6

CONCLUSION AND FUTURE WORK

6.1 Summary of Contributions

This project designed, trained, and evaluated two independent intrusion detection models: a per-flow statistical classifier based on XGBoost and a sequence-based classifier based on a Temporal Convolutional Network (TCN). The TCN operates on sequences of flows grouped by source IP and is intended to capture temporal behaviors such as beaconing activity that are difficult to observe at the individual flow level.

To enable training at the scale of the complete CTU-13 dataset, a memory-bounded processing pipeline was developed. The pipeline maintained bounded RAM consumption during chunked CSV loading, generated and consumed sequence tensors without storing the complete sequence dataset in memory, and enforced source-IP-disjoint train, validation, and test partitions to eliminate identity leakage between evaluation sets.

On held-out CTU-13 data, the XGBoost model achieved an ROC-AUC of 0.9952, while the TCN achieved an ROC-AUC of 0.9495. Although the XGBoost model achieved the higher ranking performance, the TCN obtained a substantially higher malicious-class F1-score (0.7364 compared with 0.5223), indicating that temporal sequence information provides additional detection value beyond static flow statistics.

An ensemble model combining the outputs of XGBoost and TCN was also evaluated during development. However, the ensemble achieved significantly worse performance (ROC-AUC = 0.4787 and F1-score = 0.0363) than either individual model. This result empirically justified the decision to keep both models independent and report their outputs separately rather than performing score fusion.

To evaluate cross-dataset generalization, both models were tested on the CICIDS-2017 dataset. Although direct transfer resulted in poor performance because of domain shift, a lightweight domain recalibration procedure based on feature-separability reweighting together with logistic and isotonic calibration significantly improved performance. After recalibration, the XGBoost model achieved an ROC-AUC of 0.9186 and the TCN achieved an ROC-AUC of 0.9964. Throughout this process, each model was recalibrated independently and no fusion of model outputs was performed.

6.2 Limitations

Despite the strong overall results, several limitations remain.

The malicious-class precision of the XGBoost model on CTU-13 was 0.41 at the selected operating threshold, indicating that a substantial number of false-positive alerts would be generated in a real deployment. Additional threshold optimization or downstream alert filtering would therefore be required before operational use.

Furthermore, the cross-dataset recalibration procedure is supervised and requires labeled samples from the target dataset in order to fit the logistic and isotonic calibration models. Consequently, the recalibration process would need to be repeated whenever the system is deployed in a new network environment.

Finally, the same window-level temporal features that contributed most strongly to cross-dataset transferability — including destination fan-out, destination-port entropy, and beaconing regularity — also appear to be key components of the TCN’s decision-making process. This suggests that the model may rely heavily on a relatively small subset of highly informative features, potentially limiting its robustness under changing network conditions.

6.3 Future Work

Several directions for future research emerge from this work:

1. Unsupervised or Self-Supervised Domain Adaptation

Future research could investigate domain adaptation techniques that eliminate the need for labeled target-domain data. Such approaches would improve the practicality of transferring intrusion detection models to new network environments.

2. Deployment-Oriented Threshold Optimization

The operating threshold used for XGBoost was selected according to offline evaluation metrics. Future work could explore cost-sensitive threshold selection strategies that explicitly account for analyst workload, alert budgets, and operational requirements.

3. Feature Ablation and Model Interpretability

Since several window-level structural features demonstrated stronger cross-dataset transferability than raw flow-level statistics, future work could perform feature ablation studies or attention-based analyses to identify which dimensions contribute most significantly to TCN decision making. Such analysis may lead to simpler and more transferable architectures.

REFERENCES

- [1] S. Garcia, M. Grill, J. Stiborek, and A. Zunino, “An empirical comparison of botnet detection methods,” *Computers & Security*, vol. 45, pp. 100–123, 2014.
- [2] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- [3] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” *arXiv preprint arXiv:1803.01271*, 2018.
- [4] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 2980–2988, 2017.
- [5] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116, 2018.
- [6] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.